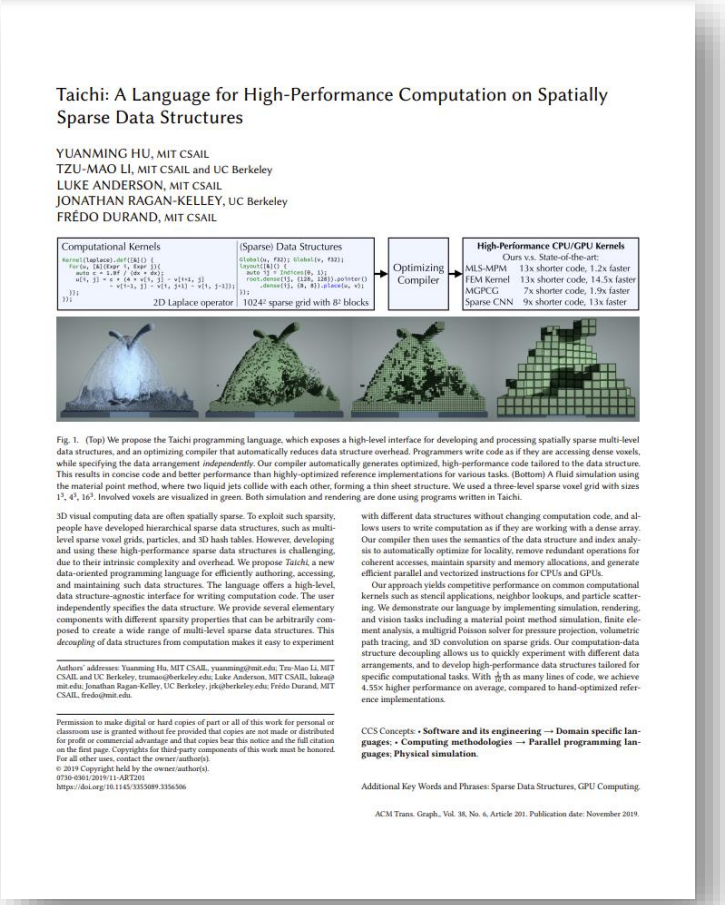
A Quick Recap of Taichi

Taichi Lang

Taichi-Lang was born in the graphics community



Taichi: A Language for High-Performance Computation on Spatially Sparse Data Structures

YUANMING HU, MIT CSAIL
TZU-MAO LI, MIT CSAIL and UC Berkeley
LUKE ANDERSON, MIT CSAIL
JONATHAN RAGAN-KELLEY, UC Berkeley
FRÉDO DURAND, MIT CSAIL

Computational Kernels

```
Kernel(Laplace).def([&]() {
  For(u, [&](Expr i, Expr j){
    auto c = 1.0f / (dx * dx);
    u[i, j] = c * (4 * v[i, j] - v[i+1, j]
                   - v[i-1, j] - v[i, j+1] - v[i, j-1]);
  });
});
```

2D Laplace operator

(Sparse) Data Structures

```
Global(u, f32); Global(v, f32);
layout([&]() {
  auto ij = Indices(0, 1);
  root.dense(ij, (128, 128)).pointer()
      .dense(ij, (8, 8)).place(u, v);
});
```

1024^2 sparse grid with 8^2 blocks

Optimizing Compiler

High-Performance CPU/GPU Kernels
Ours v.s. State-of-the-art:

MLS-MPM	13x shorter code, 1.2x faster
FEM Kernel	13x shorter code, 14.5x faster
MGPCG	7x shorter code, 1.9x faster
Sparse CNN	9x shorter code, 13x faster

Fig. 1. (Top) We propose the Taichi programming language, which exposes a high-level interface for developing and processing spatially sparse multi-level data structures, and an optimizing compiler that automatically reduces data structure overhead. Programmers write code as if they are accessing dense voxels, while specifying the data arrangement *independently*. Our compiler automatically generates optimized, high-performance code tailored to the data structure. This results in concise code and better performance than highly-optimized reference implementations for various tasks. (Bottom) A fluid simulation using the material point method, where two liquid jets collide with each other, forming a thin sheet structure. We used a three-level sparse voxel grid with sizes 1^3, 4^3, 16^3. Involved voxels are visualized in green. Both simulation and rendering are done using programs written in Taichi.

3D visual computing data are often spatially sparse. To exploit such sparsity, people have developed hierarchical sparse data structures, such as multi-level sparse voxel grids, particles, and 3D hash tables. However, developing and using these high-performance sparse data structures is challenging, due to their intrinsic complexity and overhead. We propose *Taichi*, a new data-oriented programming language for efficiently authoring, accessing, and maintaining such data structures. The language offers a high-level, data structure-agnostic interface for writing computation code. The user independently specifies the data structure. We provide several elementary components with different sparsity properties that can be arbitrarily composed to create a wide range of multi-level sparse data structures. This *decoupling* of data structures from computation makes it easy to experiment with different data structures without changing computation code, and allows users to write computation as if they are working with a dense array. Our compiler then uses the semantics of the data structure and index analysis to automatically optimize for locality, remove redundant operations for coherent accesses, maintain sparsity and memory allocations, and generate efficient parallel and vectorized instructions for CPUs and GPUs.

Our approach yields competitive performance on common computational kernels such as stencil applications, neighbor lookups, and particle scattering. We demonstrate our language by implementing simulation, rendering, and vision tasks including a material point method simulation, finite element analysis, a multigrid Poisson solver for pressure projection, volumetric path tracing, and 3D convolution on sparse grids. Our computation-data structure decoupling allows us to quickly experiment with different data arrangements, and to develop high-performance data structures tailored for specific computational tasks. With $\frac{1}{10}$th as many lines of code, we achieve 4.55× higher performance on average, compared to hand-optimized reference implementations.

Authors' addresses: Yuanming Hu, MIT CSAIL, yuanming@mit.edu; Tzu-Mao Li, MIT CSAIL and UC Berkeley, tzumao@berkeley.edu; Luke Anderson, MIT CSAIL, lukea@mit.edu; Jonathan Ragan-Kelley, UC Berkeley, jrk@berkeley.edu; Frédo Durand, MIT CSAIL, fredo@mit.edu.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
© 2019 Copyright held by the owner/author(s).
0730-0301/2019/11-ART201
https://doi.org/10.1145/3355089.3356506

CCS Concepts: • **Software and its engineering → Domain specific languages**; • **Computing methodologies → Parallel programming languages**; **Physical simulation**.

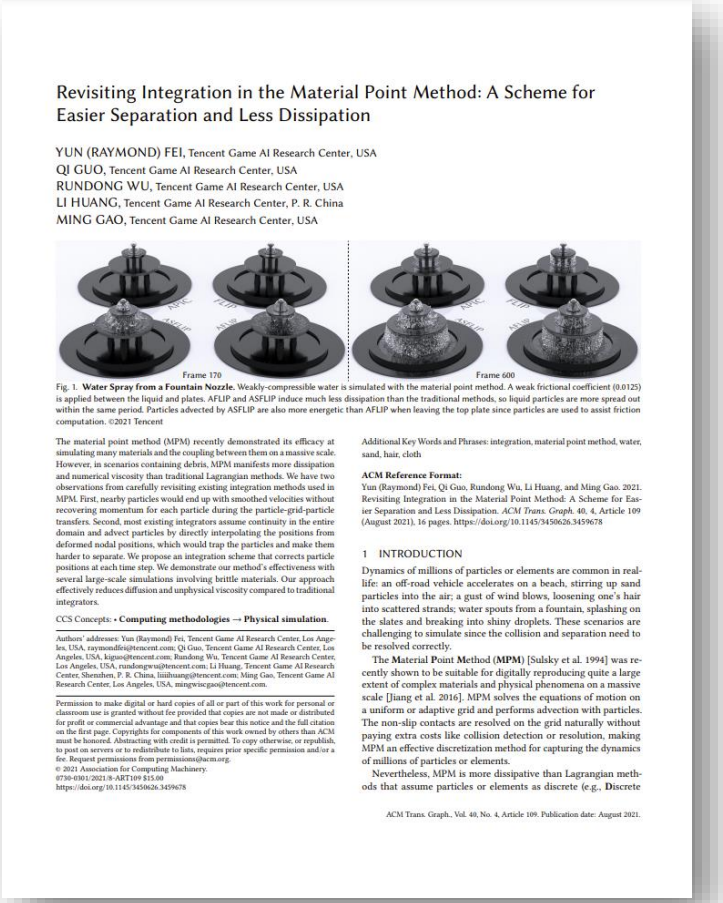
Additional Key Words and Phrases: Sparse Data Structures, GPU Computing.

ACM Trans. Graph., Vol. 38, No. 6, Article 201. Publication date: November 2019.

[Hu et al. 2019]

[Hu 2020]

Taichi-Lang serves graphics applications



Revisiting Integration in the Material Point Method: A Scheme for Easier Separation and Less Dissipation

YUN (RAYMOND) FEI, Tencent Game AI Research Center, USA
QI GUO, Tencent Game AI Research Center, USA
RUNDONG WU, Tencent Game AI Research Center, USA
LI HUANG, Tencent Game AI Research Center, P. R. China
MING GAO, Tencent Game AI Research Center, USA

Fig. 1. **Water Spray from a Fountain Nozzle.** Weakly-compressible water is simulated with the material point method. A weak frictional coefficient (0.0125) is applied between the liquid and plates. AFLIP and ASFLIP induce much less dissipation than the traditional methods, so liquid particles are more spread out within the same period. Particles advected by ASFLIP are also more energetic than AFLIP when leaving the top plate since particles are used to assist friction computation. ©2021 Tencent

The material point method (MPM) recently demonstrated its efficacy at simulating many materials and the coupling between them on a massive scale. However, in scenarios containing debris, MPM manifests more dissipation and numerical viscosity than traditional Lagrangian methods. We have two observations from carefully revisiting existing integration methods used in MPM. First, nearby particles would end up with smoothed velocities without recovering momentum for each particle during the particle-grid-particle transfers. Second, most existing integrators assume continuity in the entire domain and advect particles by directly interpolating the positions from deformed nodal positions, which would trap the particles and make them harder to separate. We propose an integration scheme that corrects particle positions at each time step. We demonstrate our method's effectiveness with several large-scale simulations involving brittle materials. Our approach effectively reduces diffusion and unphysical viscosity compared to traditional integrators.

CCS Concepts: • **Computing methodologies → Physical simulation**.

Additional Key Words and Phrases: integration, material point method, water, sand, hair, cloth

ACM Reference Format:
Yun (Raymond) Fei, Qi Guo, Rundong Wu, Li Huang, and Ming Gao. 2021. Revisiting Integration in the Material Point Method: A Scheme for Easier Separation and Less Dissipation. *ACM Trans. Graph.* 40, 4, Article 109 (August 2021), 16 pages. https://doi.org/10.1145/3450626.3459678

1 INTRODUCTION

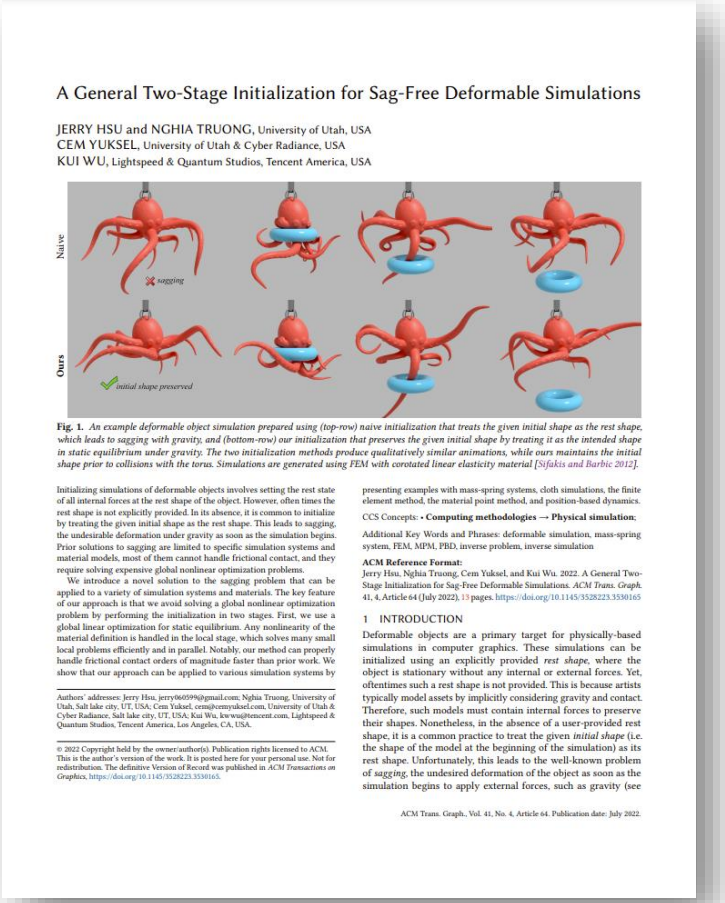
Dynamics of millions of particles or elements are common in real-life: an off-road vehicle accelerates on a beach, stirring up sand particles into the air; a gust of wind blows, loosening one's hair into scattered strands; water spouts from a fountain, splashing on the slates and breaking into shiny droplets. These scenarios are challenging to simulate since the collision and separation need to be resolved correctly.

The **M**aterial **P**oint **M**ethod (**MPM**) [Sulsky et al. 1994] was recently shown to be suitable for digitally reproducing quite a large extent of complex materials and physical phenomena on a massive scale [Jiang et al. 2016]. MPM solves the equations of motion on a uniform or adaptive grid and performs advection with particles. The non-slip contacts are resolved on the grid naturally without paying extra costs like collision detection or resolution, making MPM an effective discretization method for capturing the dynamics of millions of particles or elements.

Nevertheless, MPM is more dissipative than Lagrangian methods that assume particles or elements as discrete (e.g., **D**iscrete

ACM Trans. Graph., Vol. 40, No. 4, Article 109. Publication date: August 2021.

[Fei et al. 2021]



A General Two-Stage Initialization for Sag-Free Deformable Simulations

JERRY HSU and NGHIA TRUONG, University of Utah, USA
CEM YUKSEL, University of Utah & Cyber Radiance, USA
KUI WU, Lightspeed & Quantum Studios, Tencent America, USA

Naive
Ours
sagging
initial shape preserved

Fig. 1. *An example deformable object simulation prepared using (top-row) naive initialization that treats the given initial shape as the rest shape, which leads to sagging with gravity, and (bottom-row) our initialization that preserves the given initial shape by treating it as the intended shape in static equilibrium under gravity. The two initialization methods produce qualitatively similar animations, while ours maintains the initial shape prior to collisions with the torus. Simulations are generated using FEM with corotated linear elasticity material [Sifakis and Barbic 2012].*

Initializing simulations of deformable objects involves setting the rest state of all internal forces at the rest shape of the object. However, often times the rest shape is not explicitly provided. In its absence, it is common to initialize by treating the given initial shape as the rest shape. This leads to sagging, the undesirable deformation under gravity as soon as the simulation begins. Prior solutions to sagging are limited to specific simulation systems and material models, most of them cannot handle frictional contact, and they require solving expensive global nonlinear optimization problems.

We introduce a novel solution to the sagging problem that can be applied to a variety of simulation systems and materials. The key feature of our approach is that we avoid solving a global nonlinear optimization problem by performing the initialization in two stages. First, we use a global linear optimization for static equilibrium. Any nonlinearity of the material definition is handled in the local stage, which solves many small local problems efficiently and in parallel. Notably, our method can properly handle frictional contact orders of magnitude faster than prior work. We show that our approach can be applied to various simulation systems by presenting examples with mass-spring systems, cloth simulations, the finite element method, the material point method, and position-based dynamics.

CCS Concepts: • **Computing methodologies → Physical simulation**;

Additional Key Words and Phrases: deformable simulation, mass-spring system, FEM, MPM, PBD, inverse problem, inverse simulation

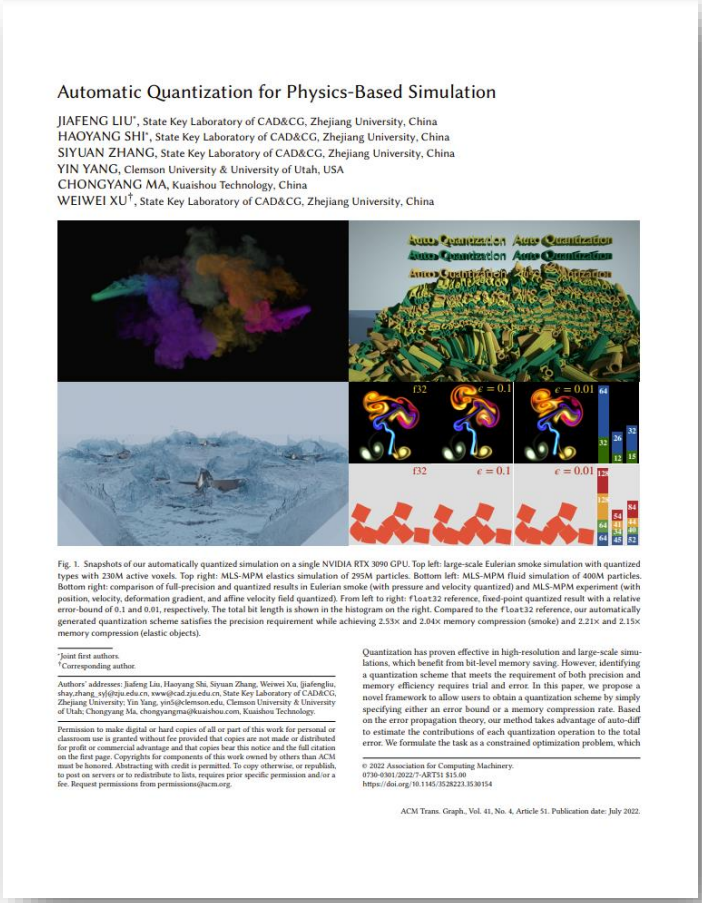
ACM Reference Format:
Jerry Hsu, Nghia Truong, Cem Yuksel, and Kui Wu. 2022. A General Two-Stage Initialization for Sag-Free Deformable Simulations. *ACM Trans. Graph.* 41, 4, Article 64 (July 2022), 13 pages. https://doi.org/10.1145/3528223.3530165

1 INTRODUCTION

Deformable objects are a primary target for physically-based simulations in computer graphics. These simulations can be initialized using an explicitly provided *rest shape*, where the object is stationary without any internal or external forces. Yet, oftentimes such a rest shape is not provided. This is because artists typically model assets by implicitly considering gravity and contact. Therefore, such models must contain internal forces to preserve their shapes. Nonetheless, in the absence of a user-provided rest shape, it is a common practice to treat the given *initial shape* (i.e. the shape of the model at the beginning of the simulation) as its rest shape. Unfortunately, this leads to the well-known problem of *sagging*, the undesired deformation of the object as soon as the simulation begins to apply external forces, such as gravity (see

ACM Trans. Graph., Vol. 41, No. 4, Article 64. Publication date: July 2022.

[Hsu et al. 2022]



Automatic Quantization for Physics-Based Simulation

JIAFENG LIU*, State Key Laboratory of CAD&CG, Zhejiang University, China
HAOYANG SHI*, State Key Laboratory of CAD&CG, Zhejiang University, China
SIYUAN ZHANG, State Key Laboratory of CAD&CG, Zhejiang University, China
YIN YANG, Clemson University & University of Utah, USA
CHONGYANG MA, Kuaishou Technology, China
WEIWEI XU†, State Key Laboratory of CAD&CG, Zhejiang University, China

Fig. 1. Snapshots of our automatically quantized simulation on a single NVIDIA RTX 3090 GPU. Top left: large-scale Eulerian smoke simulation with quantized types with 230M active voxels. Top right: MLS-MPM elastics simulation of 295M particles. Bottom left: MLS-MPM fluid simulation of 400M particles. Bottom right: comparison of full-precision and quantized results in Eulerian smoke (with pressure and velocity quantized) and MLS-MPM experiment (with position, velocity, deformation gradient, and affine velocity field quantized). From left to right: `float32` reference, fixed-point quantized result with a relative error-bound of 0.1 and 0.01, respectively. The total bit length is shown in the histogram on the right. Compared to the `float32` reference, our automatically generated quantization scheme satisfies the precision requirement while achieving 2.53× and 2.04× memory compression (smoke) and 2.21× and 2.15× memory compression (elastic objects).

Quantization has proven effective in high-resolution and large-scale simulations, which benefit from bit-level memory saving. However, identifying a quantization scheme that meets the requirement of both precision and memory efficiency requires trial and error. In this paper, we propose a novel framework to allow users to obtain a quantization scheme by simply specifying either an error bound or a memory compression rate. Based on the error propagation theory, our method takes advantage of auto-diff to estimate the contributions of each quantization operation to the total error. We formulate the task as a constrained optimization problem, which

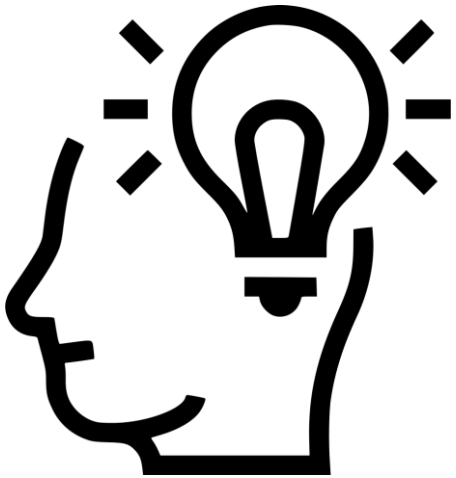
ACM Trans. Graph., Vol. 41, No. 4, Article 51. Publication date: July 2022.

[Liu et al. 2022]

The Taichi programming language

SIGGRAPH ASIA 2022 DAEGU

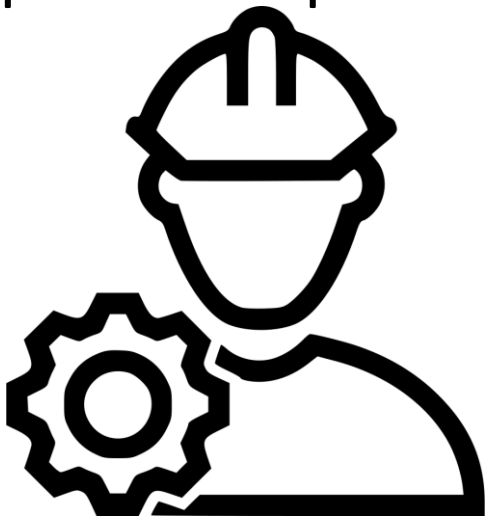
- A domain specific language (DSL) for computer graphics and parallel computing. The Taichi-lang…
  - has a Python frontend;
  - is Optimized for parallel computing;
  - can be deployed everywhere on CPUs and GPUs;
  - thrives through open-source development.



The Taichi programming language

SIGGRAPH ASIA 2022 DAEGU

- A domain specific language (DSL) for computer graphics and parallel computing. The Taichi-lang…
  - has a Python frontend;
  - is Optimized for parallel computing;
  - can be deployed everywhere on CPUs and GPUs;
  - thrives through open-source development.



The Taichi programming language

SIGGRAPH ASIA 2022 DAEGU

- A domain specific language (DSL) for computer graphics and parallel computing. The Taichi-lang…
  - has a Python frontend;
  - is Optimized for parallel computing;
  - can be deployed everywhere on CPUs and GPUs;
  - thrives through open-source development.

The Taichi programming language

SIGGRAPH ASIA 2022 DAEGU

- A domain specific language (DSL) for computer graphics and parallel computing. The Taichi-lang…
  - has a Python frontend;
  - is Optimized for parallel computing;
  - can be deployed everywhere on CPUs and GPUs;
  - thrives through open-source development.

Taichi

Installation

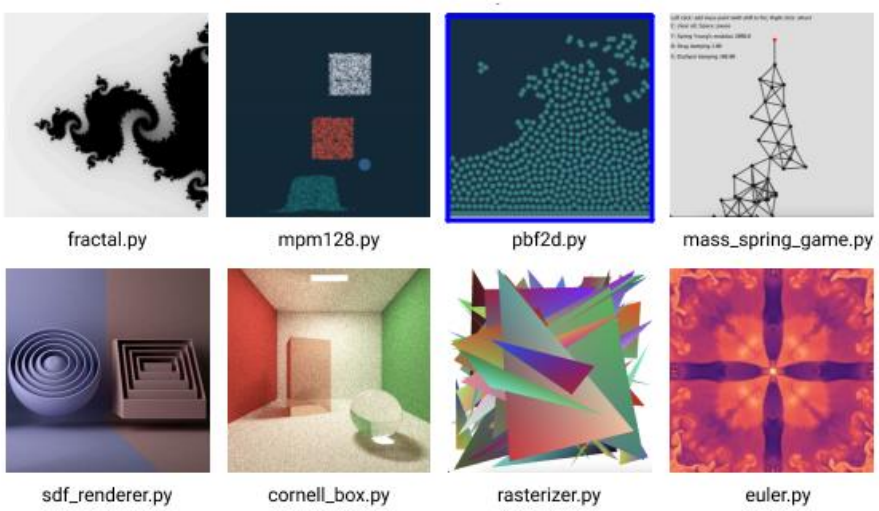
- python3 -m **pip install taichi** -U

- Until today (10/30/2022)
  - Latest version: 1.2.0
  - Taichi supports Python 3.6/3.7/3.8/3.9/3.10 (64-bit)
  - Taichi supports Windows, Linux, and OS X.

Quick start

- Use python3 –m taichi or simply ti to start Taichi's CLI

- For example:
  - ti gallery: list featured examples / ti example: list all provided examples



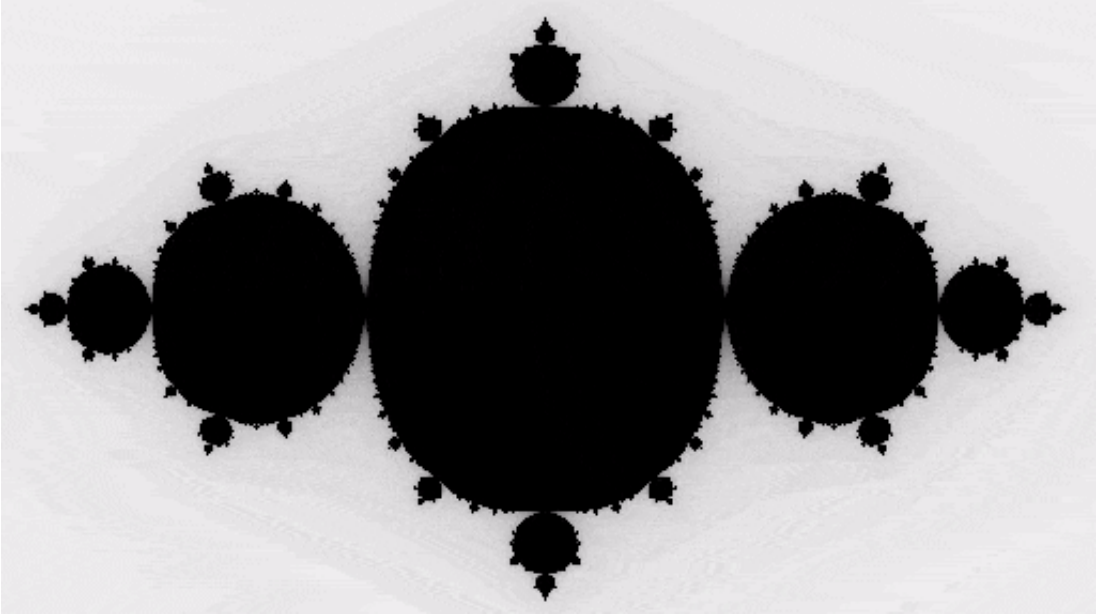

```
                    TAICHI EXAMPLES
ad_gravity              keyboard                patterns
comet                   laplace                 pbf2d
cornell_box             mandelbrot_zoom         physarum
diff_sph                marching_squares        print_offset
euler                   mass_spring_3d_ggui     rasterizer
explicit_activation     mass_spring_game        regression
export_mesh             mass_spring_game_ggui   sdf_renderer
export_ply              mciso_advanced          simple_derivative
export_videos           mgpcg                   simple_texture
fem128                  mgpcg_advanced          simple_uv
fem128_ggui             minimal                 stable_fluid
fem99                   minimization            stable_fluid_ggui
fractal                 mpm128                  stable_fluid_graph
fractal3d_ggui          mpm128_ggui             taichi_bitmasked
fullscreen              mpm3d                   taichi_dynamic
game_of_life            mpm3d_ggui              taichi_logo
gui_image_io            mpm88                   taichi_sparse
gui_widgets             mpm88_graph             tutorial
implicit_fem            mpm99                   vortex_rings
implicit_mass_spring    mpm_lagrangian_forces   waterwave
inital_value_problem    nbody
initial_value_problem   odop_solar
usage: ti example [-h] [-p] [-P] [-s] name
```

Let’s try: a simple Julia set program

SIGGRAPH ASIA 2022 DAEGU

```
import taichi as ti

ti.init(arch=ti.gpu) # Run on GPU by default

n = 320
pixels = ti.field(dtype=float, shape=(n * 2, n))

@ti.func
def complex_sqr(z):
    return ti.Vector([z[0]**2 - z[1]**2, z[1] * z[0] * 2])

@ti.kernel
def paint(t: float):
    for i, j in pixels:  # Parallized over all pixels
        c = ti.Vector([-0.8, ti.cos(t) * 0.2])
        z = ti.Vector([i / n - 1, j / n - 0.5]) * 2
        iterations = 0
        while z.norm() < 20 and iterations < 50:
            z = complex_sqr(z) + c
            iterations += 1
        pixels[i, j] = 1 - iterations * 0.02

gui = ti.GUI("Julia Set", res=(n * 2, n))

for i in range(1000000):
    paint(i * 0.03)
    gui.set_image(pixels)
    gui.show()
```

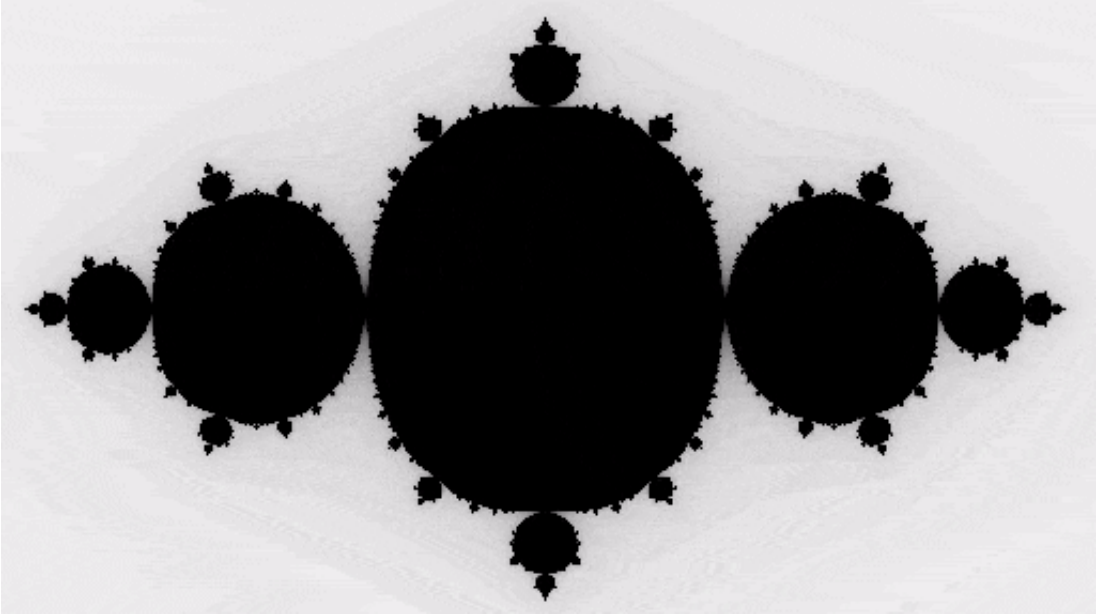
- $z = z^2 + c$



“import taichi as ti”

```
import taichi as ti

ti.init(arch=ti.gpu) # Run on GPU by default

n = 320
pixels = ti.field(dtype=float, shape=(n * 2, n))

@ti.func
def complex_sqr(z):
    return ti.Vector([z[0]**2 - z[1]**2, z[1] * z[0] * 2])

@ti.kernel
def paint(t: float):
    for i, j in pixels:  # Parallized over all pixels
        c = ti.Vector([-0.8, ti.cos(t) * 0.2])
        z = ti.Vector([i / n - 1, j / n - 0.5]) * 2
        iterations = 0
        while z.norm() < 20 and iterations < 50:
            z = complex_sqr(z) + c
            iterations += 1
        pixels[i, j] = 1 - iterations * 0.02

gui = ti.GUI("Julia Set", res=(n * 2, n))

for i in range(1000000):
    paint(i * 0.03)
    gui.set_image(pixels)
    gui.show()
```

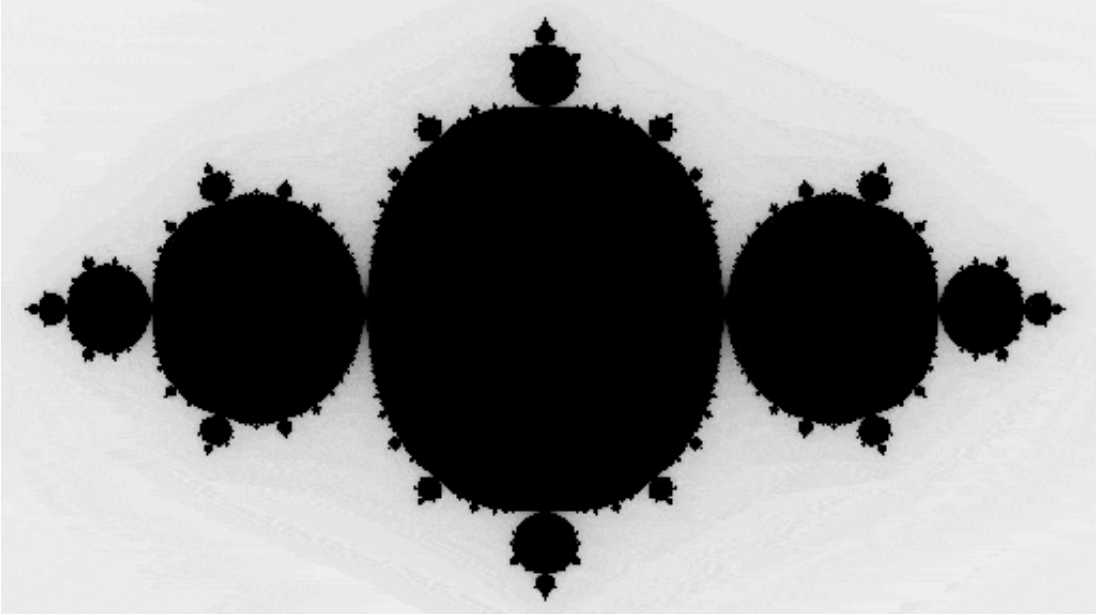
- ti.init()
  - The entry point of every Taichi program



The most useful data container: ti.field

```python
import taichi as ti

ti.init(arch=ti.gpu) # Run on GPU by default

n = 320
pixels = ti.field(dtype=float, shape=(n * 2, n))

@ti.func
def complex_sqr(z):
    return ti.Vector([z[0]**2 - z[1]**2, z[1] * z[0] * 2])

@ti.kernel
def paint(t: float):
    for i, j in pixels:  # Parallized over all pixels
        c = ti.Vector([-0.8, ti.cos(t) * 0.2])
        z = ti.Vector([i / n - 1, j / n - 0.5]) * 2
        iterations = 0
        while z.norm() < 20 and iterations < 50:
            z = complex_sqr(z) + c
            iterations += 1
        pixels[i, j] = 1 - iterations * 0.02

gui = ti.GUI("Julia Set", res=(n * 2, n))

for i in range(1000000):
    paint(i * 0.03)
    gui.set_image(pixels)
    gui.show()
```

- multi-dimensional array of elements
- like an ndarray in *NumPy* or a tensor in *PyTorch*



ti.field

- “a global N-d array of elements”

```
heat_field = ti.field(dtype=ti.f32, shape=(256, 256))
```

ti.field

- "a global N-d array of elements"
  - global: can be read/written from both the Taichi-scope and the Python-scope
  - N-d: (Scalar: N=0), (Vector: N=1), (Matrix: N=2), (N = 3, 4, 5, …)
  - elements: scalar, vector, matrix, struct

ti.field

- "a global N-d array of elements"
  - global: can be read/written from both the Taichi-scope and the Python-scope
  - N-d: (Scalar: N=0), (Vector: N=1), (Matrix: N=2), (N = 3, 4, 5, …)
  - elements: scalar, vector, matrix, struct

ti.field

- "a global N-d array of elements"
  - global: can be read/written from both the Taichi-scope and the Python-scope
  - N-d: (Scalar: N=0), (Vector: N=1), (Matrix: N=2), (N = 3, 4, 5, …)
  - elements: scalar, vector, matrix, struct

ti.field

- "a global N-d array of elements"
  - global: can be read/written from both the Taichi-scope and the Python-scope
  - N-d: (Scalar: N=0), (Vector: N=1), (Matrix: N=2), (N = 3, 4, 5, …)
  - elements: scalar, vector, matrix, struct
  - access elements in a field using [i,j,k,…] indexing

```
import taichi as ti
ti.init()

pixels = ti.field(dtype=float, shape=(16, 8))

pixels[1, 2] = 42.0
```

```
import taichi as ti
ti.init()

vf = ti.Vector.field(3, ti.f32, shape=4)

@ti.kernel
def foo():
    v = ti.Vector([1, 2, 3])
    vf[0] = v
```

ti.field

- "a global N-d array of elements"
  - global: can be read/written from both the Taichi-scope and the Python-scope
  - N-d: (Scalar: N=0), (Vector: N=1), (Matrix: N=2), (N = 3, 4, 5, …)
  - elements: scalar, vector, matrix, struct
  - access elements in a field using [i,j,k,…] indexing
    - One special case, access a zero-d field using [None]

```
zero_d_scalar = ti.field(ti.f32, shape=())
zero_d_scalar[None] = 1.0

zero_d_vec = ti.Vector.field(2, ti.f32, shape=())
zero_d_vec[None] = ti.Vector([2.0, 2.5])
```

ti.field examples

- “3D gravitational field in a 256x256x128 room”

```
gravitational_field = ti.Vector.field(n = 3,dtype=ti.f32,shape=(256,256,128))
```

- “2D strain-tensor field in a 64x64 grid”

```
strain_tensor_field = ti.Matrix.field(n = 2,m = 2, dtype=ti.f32, shape=(64,64))
```

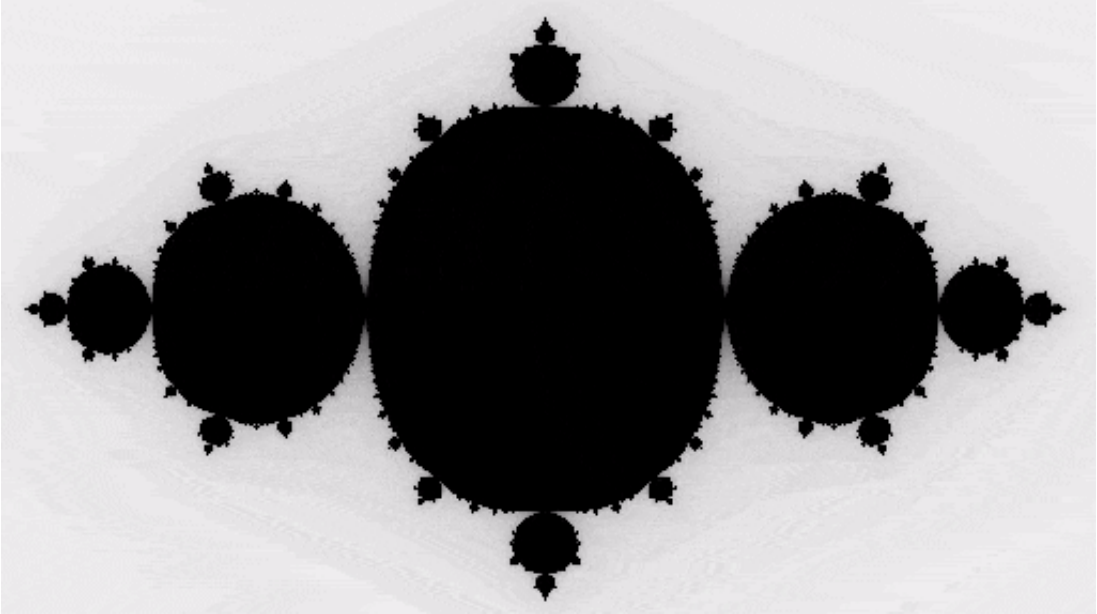
- “a global scalar that I want to access in a Taichi kernel”

```
global_scalar = ti.field(dtype=ti.f32, shape=())
```

Computations in Taichi

```python
import taichi as ti

ti.init(arch=ti.gpu) # Run on GPU by default

n = 320
pixels = ti.field(dtype=float, shape=(n * 2, n))

@ti.func
def complex_sqr(z):
    return ti.Vector([z[0]**2 - z[1]**2, z[1] * z[0] * 2])

@ti.kernel
def paint(t: float):
    for i, j in pixels:  # Parallized over all pixels
        c = ti.Vector([-0.8, ti.cos(t) * 0.2])
        z = ti.Vector([i / n - 1, j / n - 0.5]) * 2
        iterations = 0
        while z.norm() < 20 and iterations < 50:
            z = complex_sqr(z) + c
            iterations += 1
        pixels[i, j] = 1 - iterations * 0.02

gui = ti.GUI("Julia Set", res=(n * 2, n))

for i in range(1000000):
    paint(i * 0.03)
    gui.set_image(pixels)
    gui.show()
```

- Decorated with @ti.kernel or @ti.func
- The outermost for loop is automatically parallelized



For-loops in a @ti.kernel

SIGGRAPH ASIA 2022 DAEGU

- For loops at the **outermost scope** in a Taichi kernel is **automatically parallelized**

```
@ti.kernel
def fill():
    for i in range(10): # Parallelized
        x[i] += i

        s = 0
        for j in range(5): # Serialized in each parallel thread
            s += j

        y[i] = s

    for k in rang(20): # Parallelized
        z[k] = k
```

For-loops in a @ti.kernel

- Outermost scope ?

```
import taichi as ti
ti.init()

@ti.kernel
def foo(k: ti.i32):
    for i in range(10): # Parallelized :-)
        if k > 42:
            ...

@ti.kernel
def bar(k: ti.i32):
    if k > 42:
        for i in range(10): # Serial :-(
            ...
```

Types of for-loops in Taichi

SIGGRAPH ASIA 2022 DAEGU

- range-for: loops over a **range**, identical to Python range-for
- struct-for: loops over a **ti.field**, only lives at the outermost scope

```
import taichi as ti
ti.init()

N = 10
x = ti.field(dtype=ti.i32, shape=N)

@ti.kernel
def foo():
  for i in range(N):
      x[i] = i

foo()
```

```
import taichi as ti
ti.init()

N = 10
x = ti.Vector.field(2,dtype=ti.i32, shape=(N,N))

@ti.kernel
def foo():
  for i,j in x:
      x[i,j] = ti.Vector([i, j])

foo()
```

Types of for-loops in Taichi

SIGGRAPH ASIA 2022 DAEGU

- range-for: loops over a **range**, identical to Python range-for
- struct-for: loops over a **ti.field**, only lives at the outermost scope

```
import taichi as ti
ti.init()

N = 10
x = ti.field(dtype=ti.i32, shape=N)

@ti.kernel
def foo():
  for i in range(N):
      x[i] = i

foo()
```

```
import taichi as ti
ti.init()

N = 10
x = ti.Vector.field(2,dtype=ti.i32, shape=(N,N))

@ti.kernel
def foo():
  for i,j in x:
      x[i,j] = ti.Vector([i, j])

foo()
```

Race condition

SIGGRAPH ASIA 2022 DAEGU

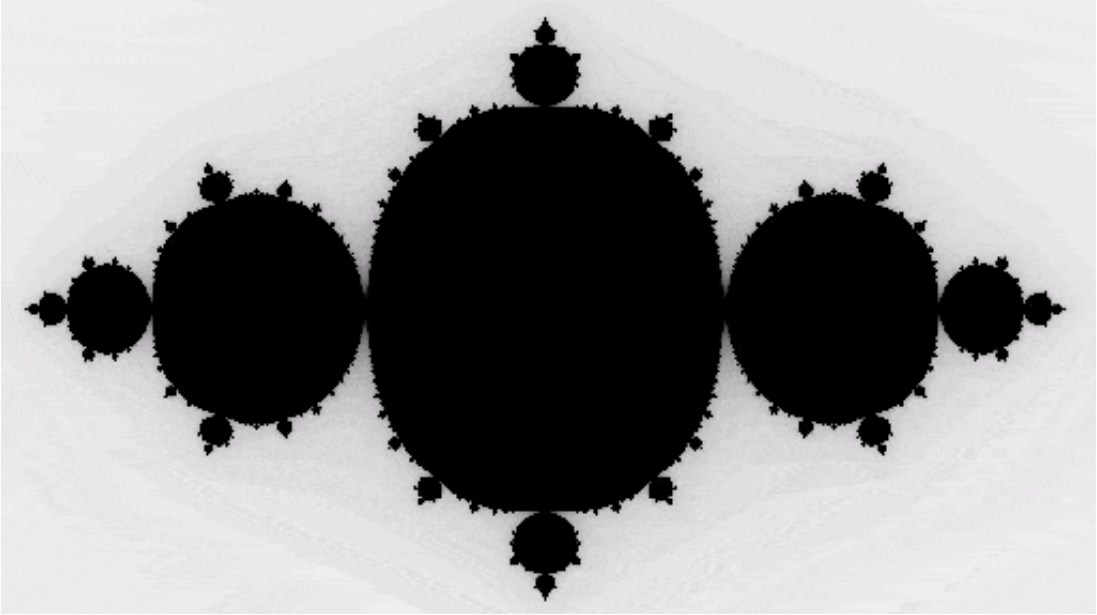
- Taichi uses += as an atomic add
- The compiler optimizes for unnecessary atomic operations

```
@ti.kernel
def sum():
    for i in range(10):
        # 1. OK
        total[None] += x[i]

        # 2. OK
        ti.atomic_add(total[None], x[i])

        # 3. data race
        total[None] = total[None] + x[i]
```

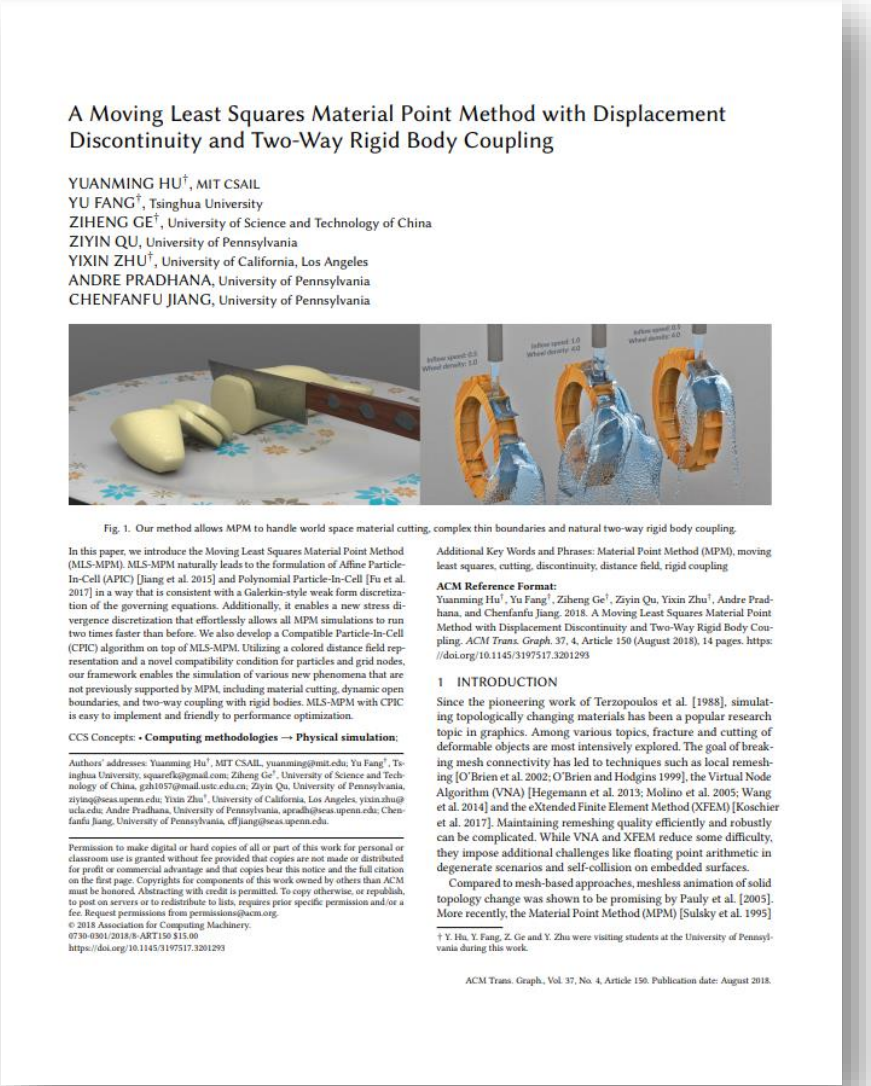
Visualization in Taichi

```python
import taichi as ti

ti.init(arch=ti.gpu) # Run on GPU by default

n = 320
pixels = ti.field(dtype=float, shape=(n * 2, n))

@ti.func
def complex_sqr(z):
    return ti.Vector([z[0]**2 - z[1]**2, z[1] * z[0] * 2])

@ti.kernel
def paint(t: float):
    for i, j in pixels:  # Parallized over all pixels
        c = ti.Vector([-0.8, ti.cos(t) * 0.2])
        z = ti.Vector([i / n - 1, j / n - 0.5]) * 2
        iterations = 0
        while z.norm() < 20 and iterations < 50:
            z = complex_sqr(z) + c
            iterations += 1
        pixels[i, j] = 1 - iterations * 0.02

gui = ti.GUI("Julia Set", res=(n * 2, n))

for i in range(1000000):
    paint(i * 0.03)
    gui.set_image(pixels)
    gui.show()
```

- Using the GUI or GGUI system to visualize your results



“ti example mpm128” [Hu et al. 2018]



A Moving Least Squares Material Point Method with Displacement Discontinuity and Two-Way Rigid Body Coupling

YUANMING HU†, MIT CSAIL
YU FANG†, Tsinghua University
ZIHENG GE†, University of Science and Technology of China
ZIYIN QU, University of Pennsylvania
YIXIN ZHU†, University of California, Los Angeles
ANDRE PRADHANA, University of Pennsylvania
CHENFANFU JIANG, University of Pennsylvania

Fig. 1. Our method allows MPM to handle world space material cutting, complex thin boundaries and natural two-way rigid body coupling.

In this paper, we introduce the Moving Least Squares Material Point Method (MLS-MPM). MLS-MPM naturally leads to the formulation of Affine Particle-In-Cell (APIC) [Jiang et al. 2015] and Polynomial Particle-In-Cell [Fu et al. 2017] in a way that is consistent with a Galerkin-style weak form discretization of the governing equations. Additionally, it enables a new stress divergence discretization that effortlessly allows all MPM simulations to run two times faster than before. We also develop a Compatible Particle-In-Cell (CPIC) algorithm on top of MLS-MPM. Utilizing a colored distance field representation and a novel compatibility condition for particles and grid nodes, our framework enables the simulation of various new phenomena that are not previously supported by MPM, including material cutting, dynamic open boundaries, and two-way coupling with rigid bodies. MLS-MPM with CPIC is easy to implement and friendly to performance optimization.

CCS Concepts: • **Computing methodologies → Physical simulation**;

Authors' addresses: Yuanming Hu†, MIT CSAIL, yuanming@mit.edu; Yu Fang†, Tsinghua University, squarefk@gmail.com; Ziheng Ge†, University of Science and Technology of China, gzh1057@mail.ustc.edu.cn; Ziyin Qu, University of Pennsylvania, ziyinq@seas.upenn.edu; Yixin Zhu†, University of California, Los Angeles, yixin.zhu@ucla.edu; Andre Pradhana, University of Pennsylvania, apradh@seas.upenn.edu; Chenfanfu Jiang, University of Pennsylvania, cffjiang@seas.upenn.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2018 Association for Computing Machinery.

0730-0301/2018/8-ART150 $15.00
https://doi.org/10.1145/3197517.3201293

Additional Key Words and Phrases: Material Point Method (MPM), moving least squares, cutting, discontinuity, distance field, rigid coupling

ACM Reference Format:
Yuanming Hu†, Yu Fang†, Ziheng Ge†, Ziyin Qu, Yixin Zhu†, Andre Pradhana, and Chenfanfu Jiang. 2018. A Moving Least Squares Material Point Method with Displacement Discontinuity and Two-Way Rigid Body Coupling. *ACM Trans. Graph.* 37, 4, Article 150 (August 2018), 14 pages. https://doi.org/10.1145/3197517.3201293

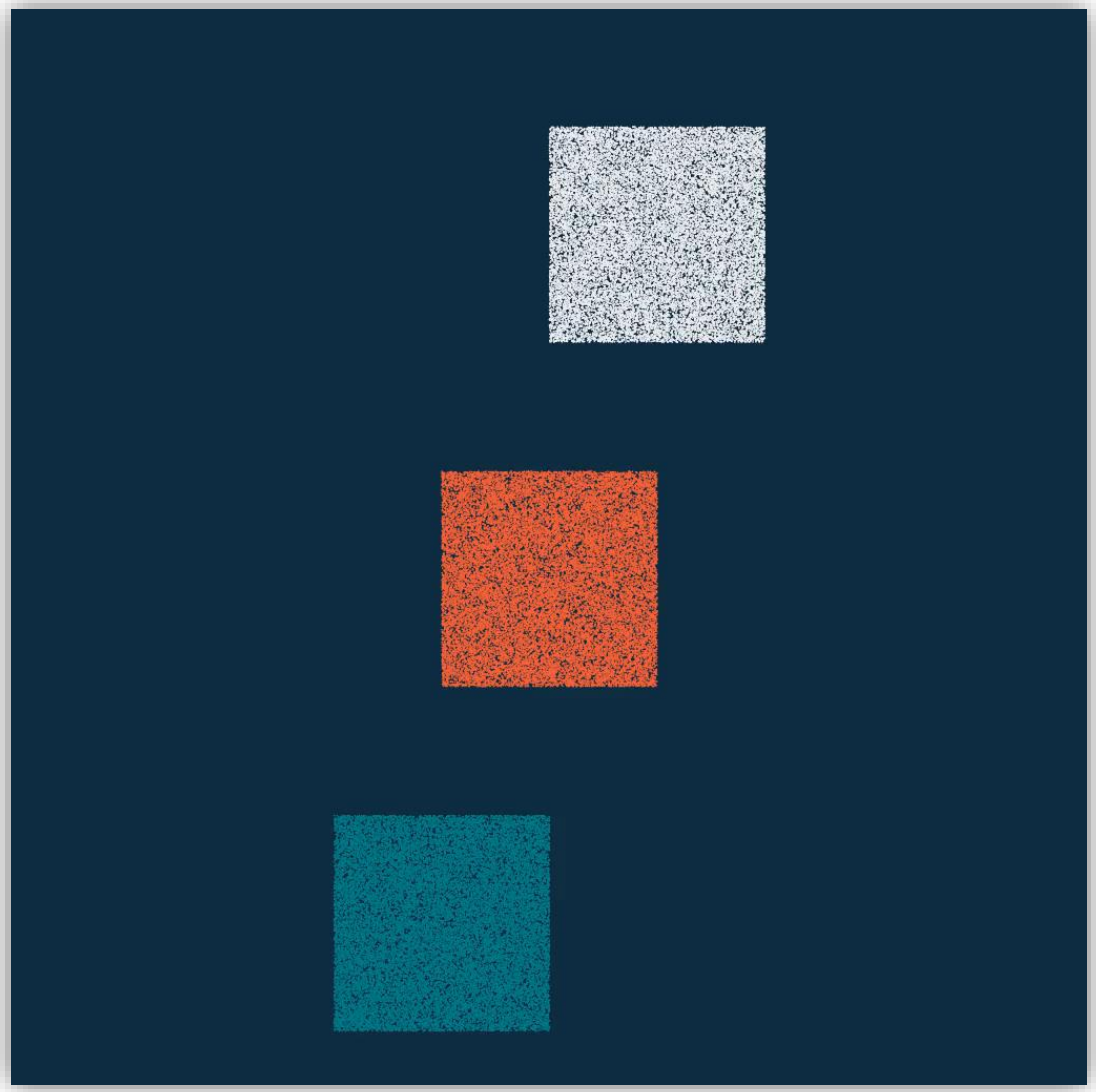
1 INTRODUCTION

Since the pioneering work of Terzopoulos et al. [1988], simulating topologically changing materials has been a popular research topic in graphics. Among various topics, fracture and cutting of deformable objects are most intensively explored. The goal of breaking mesh connectivity has led to techniques such as local remeshing [O'Brien et al. 2002; O'Brien and Hodgins 1999], the Virtual Node Algorithm (VNA) [Hegemann et al. 2013; Molino et al. 2005; Wang et al. 2014] and the eXtended Finite Element Method (XFEM) [Koschier et al. 2017]. Maintaining remeshing quality efficiently and robustly can be complicated. While VNA and XFEM reduce some difficulty, they impose additional challenges like floating point arithmetic in degenerate scenarios and self-collision on embedded surfaces.

Compared to mesh-based approaches, meshless animation of solid topology change was shown to be promising by Pauly et al. [2005]. More recently, the Material Point Method (MPM) [Sulsky et al. 1995]

† Y. Hu, Y. Fang, Z. Ge and Y. Zhu were visiting students at the University of Pennsylvania during this work.

ACM Trans. Graph., Vol. 37, No. 4, Article 150. Publication date: August 2018.



Remark

- The most useful data container:
  - ti.field
- Functions taken over by the Taichi compiler:
  - decorated with @ti.kernel or @ti.func
- For-loops
  - The outer-most for-loop in a @ti.kernel is parallelized
  - Types of for-loops: range-for and struct-for